

# Complexity analysis of the RTIGER core optimizations

A per-step big- $\mathcal{O}$  accounting of the `optimize-julia-core` fork

Gemma Lab (`faustovrz/RTIGER`)

June 10, 2026

## Abstract

This note derives, step by step, the asymptotic cost of the rigid-HMM (rHMM) Baum–Welch fit that RTIGER performs, and the big- $\mathcal{O}$  improvement contributed by each optimization in the `optimize-julia-core` fork. Equation numbers refer to the RTIGER supplementary text. Every optimization preserves the arithmetic of the model (Sections 2–4 of the supplement); only the cost of evaluating it changes. Two of the optimizations remove an asymptotic factor (the rigidity window  $r$ , and the marker count  $T$  in the emission line search), one removes a memory factor (the sample count  $N$ ), and two are constant-factor (allocation) wins that leave the complexity class unchanged.

## 1 Notation and cost model

| symbol    | meaning                                                                              |
|-----------|--------------------------------------------------------------------------------------|
| $T$       | markers (positions) in one observation chain                                         |
| $C$       | number of chains (samples $\times$ chromosomes); $T_{\text{tot}} = \sum_c T_c$       |
| $N$       | number of samples (so $C$ and $T_{\text{tot}}$ grow $\propto N$ )                    |
| $ S  = s$ | number of hidden states (= 3: paternal / heterozygous / maternal)                    |
| $r$       | rigidity; augmented rigid state space $S' = S \times \{1, \dots, r\}$ , $ S'  = s r$ |
| $D$       | number of <i>distinct</i> emission pairs $(k, n)$ in a chain                         |
| $E$       | objective evaluations per state in one emission-M-step line search                   |

Table 1: Cost parameters. For low-coverage genotyping data  $D \ll T$ : the read depth  $n$  is small, so the distinct  $(k, n)$  pairs number a few dozen regardless of  $T$ .

The emission is a Beta–Binomial (supplement §3.3),  $\psi_s(k; n, a_s, b_s) = \binom{n}{k} \text{B}(k+a_s, n-k+b_s) / \text{B}(a_s, b_s)$ , whose evaluation costs a handful of `lgamma` calls — the dominant scalar op of the fit. A full fit is

$$\text{cost} = (\text{\#EM iterations}) \times \sum_{c=1}^C (\text{E-step}_c + \text{M-step}_c),$$

so we analyse the per-iteration, per-chain cost and then note how it aggregates.

## 2 The auxiliary window product $\Psi_k^t$ (productpsi)

The rigid recursions repeatedly need the windowed emission product (supplement Eq. 21)

$$\Psi_k^t = \prod_{u=\max(t-r+1,1)}^t \psi_k(o_u), \quad 1 \leq t \leq T. \quad (21)$$

Computing each of the  $T \cdot s$  values by re-multiplying its length- $r$  window costs  $\mathcal{O}(Tsr)$ . In log space  $\log \Psi_k^t = \sum_u \log \psi_k(o_u)$  is a fixed-width sliding sum: maintain a running window, add the entering term and drop the leaving term in  $\mathcal{O}(1)$  per step.

$$\boxed{\mathcal{O}(Tsr) \rightarrow \mathcal{O}(Ts)} \quad (\text{removes the rigidity factor } r; \text{ commit d725651}).$$

At the production rigidity  $r = 250$  this is a  $\sim r$ -fold reduction of the  $\Psi$  term.

## 3 Emission log-pdf memoization (getlogpsi)

Each forward, backward and Viterbi step evaluates  $\psi_s(o_t)$  at every position (Eqs. 22–27, 75). Done literally, that is  $\mathcal{O}(Ts)$  Beta–Binomial evaluations per iteration — each one several `lgamma` calls. But  $\psi_s(o_t)$  depends on  $o_t = (k_t, n_t)$  only, and there are only  $D$  distinct pairs. Memoizing  $\log \psi_s$  over the distinct  $(k, n)$  replaces the expensive evaluations with cheap table lookups:

$$\boxed{\mathcal{O}(Ts) \text{ Beta–Binomial evals} \rightarrow \mathcal{O}(Ds) \text{ evals} + \mathcal{O}(Ts) \text{ lookups}}$$

i.e. the costly part shrinks by the factor  $T/D$  (commit d725651).

## 4 Emission M-step line search (the dominant cost)

The Beta–Binomial parameters are fit by a 1-D line search over  $\tau_s$  that maximizes (supplement Eqs. 38, 43)

$$Q(\tau_s) \propto \sum_{c=1}^C \sum_{t=1}^{T_c} \gamma_s^{t,c} \log \psi_s(k_t^c; n_t^c, a_s = m_s \tau_s, b_s = (1 - m_s) \tau_s). \quad (43)$$

**Original.** Each of the  $E$  line-search evaluations rebuilds a Beta–Binomial object and sums  $\log \psi_s$  over all  $T_{\text{tot}}$  markers:  $\mathcal{O}(EsT_{\text{tot}})$  Beta–Binomial constructions per iteration. The supplement notes the forward/backward/Viterbi kernels are the “time-critical” parts; in practice this repeated full-marker objective is what dominates — empirically  $\sim 99.97\%$  of upstream runtime.

**Optimized.** Because the sum only depends on the data through the  $\gamma$ -weighted counts at each distinct pair, regroup it once:

$$Q(\tau_s) = \sum_{(k,n) \in \mathcal{D}} \underbrace{\left( \sum_{t: (k_t, n_t) = (k,n)} \gamma_s^t \right)}_{G_s(k,n) \text{ (pre-summed once)}} \log \psi_s(k; n, a_s, b_s). \quad (1)$$

The weights  $G_s(k, n)$  are accumulated in one  $\mathcal{O}(sT_{\text{tot}})$  pass; each line-search evaluation then costs  $\mathcal{O}(sD)$ :

$$\boxed{\mathcal{O}(EsT_{\text{tot}}) \rightarrow \mathcal{O}(sT_{\text{tot}} + EsD)}$$

Since  $ED \ll T_{\text{tot}}$ , the M-step collapses from “ $E$  passes over all markers, rebuilding objects” to “one binning pass” — a measured  $\sim 1500\times$  at production scale (commit 16b8a65). After this, the M-step is no longer the bottleneck.

## 5 Forward / backward and Viterbi (constant-factor)

The rigid forward (Eqs. 22–23), backward (Eqs. 26–27) and Viterbi (Eq. 75)

$$\varphi_k^t = \max_{j \in S} \begin{cases} \psi_k(o_t) a_{kk} \varphi_k^{t-1} & j = k \\ \Psi_k^t a_{jk} \varphi_j^{t-r} & j \neq k \end{cases} \quad (75)$$

each take a max/sum over  $s$  predecessors at every position, giving  $\mathcal{O}(T s^2)$  per chain (the dwell-counter shifts for  $k < r$  are  $\mathcal{O}(1)$  copies once  $\Psi$  is available). The optimizations here — in-place scalar log-sum-exp (Eq. 80) and an allocation-free in-place arg-max — do *not* change the class:

$$\mathcal{O}(T s^2) \longrightarrow \mathcal{O}(T s^2), \quad \text{constant-factor: } \sim 5 \times (\text{fwd/bwd, commit 44ed85b}), \sim 12 \times (\text{Viterbi, 6cf85ca}),$$

with  $\sim 90 \times$  fewer allocations in the forward/backward kernel. Once the emission M-step is fixed (§4), these  $\mathcal{O}(T s^2)$  kernels are what the per-iteration cost reduces to — hence *linear in  $T$* .

## 6 Memory: streaming the M-step (eb79933)

The M-step needs the data only through pooled sufficient statistics that are *sums* over chains (Eqs. 40–42):  $\sum_c \zeta$ ,  $\sum_c \gamma^1$ , and the  $\gamma$ -weighted counts  $G_s(k, n)$ .

**Original.** `EM()` held, for *all*  $C$  chains at once, the per-chain arrays  $\zeta$  ( $T \times s^2$ ),  $\gamma$  ( $s \times T$ ) and  $\alpha, \beta, \psi$  ( $sr \times T$ ), pooling only at the end:

$$\text{peak} = \mathcal{O}\left(\sum_c T_c (s^2 + sr)\right) = \mathcal{O}(N T (s^2 + sr)) \quad (\text{linear in } N).$$

**Streaming.** Fold each chain’s contribution into the running sums inside the E-step loop and discard its arrays immediately (the public `@Probabilities` slot, which nothing reads, is no longer filled):

$$\boxed{\mathcal{O}(N T (s^2 + sr)) \longrightarrow \mathcal{O}(T_{\max} (s^2 + sr)) + \mathcal{O}(T_{\text{tot}})}$$

peak scratch becomes one chain’s worth, *independent of  $N$* ; the only  $N$ -linear residual is the input count matrices that must be held. Measured:  $\sim 33$  GB projected  $\rightarrow \sim 3.6$  GB at  $1400 \times 50k$ .

## 7 Summary

## 8 Reconciliation with the measured scaling

A marker-scaling head-to-head (one shared 109,703-locus panel, decimated by odd index to five sizes, both cores run to convergence) gives empirical per-iteration exponents

$$\text{original} \approx \text{markers}^{2.1}, \quad \text{optimized} \approx \text{markers}^{1.1}.$$

These are consistent with the analysis:

- **Optimized.** With the M-step reduced to a binning pass plus an  $\mathcal{O}(EsD)$  search ( $D$  nearly flat), the per-iteration cost is dominated by the  $\mathcal{O}(T s^2)$  forward/backward/Viterbi kernels — *linear in  $T$* , matching the measured  $\sim 1.1$  exponent.

| optimization                                | before (per iter)               | after (per iter)                      | gain                     |
|---------------------------------------------|---------------------------------|---------------------------------------|--------------------------|
| $\Psi$ window ( <code>productpsi</code> )   | $\mathcal{O}(Tsr)$              | $\mathcal{O}(Ts)$                     | drop $r$                 |
| emission log-pdf ( <code>getlogpsi</code> ) | $\mathcal{O}(Ts)$ evals         | $\mathcal{O}(Ds)$ evals               | $T/D$                    |
| emission M-step ( <code>16b8a65</code> )    | $\mathcal{O}(EsT_{\text{tot}})$ | $\mathcal{O}(sT_{\text{tot}} + EsD)$  | $\sim 1500\times$        |
| forward/backward ( <code>44ed85b</code> )   | $\mathcal{O}(Ts^2)$             | $\mathcal{O}(Ts^2)$                   | $\sim 5\times$ (const.)  |
| Viterbi ( <code>6cf85ca</code> )            | $\mathcal{O}(Ts^2)$             | $\mathcal{O}(Ts^2)$                   | $\sim 12\times$ (const.) |
| memory, streaming ( <code>eb79933</code> )  | $\mathcal{O}(NT(s^2+sr))$       | $\mathcal{O}(T_{\text{max}}(s^2+sr))$ | drop $N$                 |

Table 2: Per-EM-iteration time complexity (top) and peak-memory complexity (bottom).  $s = 3$  is constant; the asymptotically meaningful factors are  $T$ ,  $r$ ,  $D$ ,  $E$  and  $N$ .

- **Original.** The per-iteration cost is dominated by the emission line search  $\mathcal{O}(EsT_{\text{tot}})$ , but  $E$  (the number of `Optim` objective evaluations) is data-dependent and grows with scale, and the Beta-Binomial object is rebuilt per marker. The result is an empirically super-linear, erratic per-iteration cost ( $\sim \text{markers}^{2.1}$ , with iteration-to-iteration spikes) rather than a clean  $\mathcal{O}(T)$  — exactly the behaviour the grouping in §4 removes.

The net effect: per-iteration cost goes from empirically  $\sim$ quadratic and erratic to provably linear ( $\mathcal{O}(Ts^2)$ ), and the optimized-vs-original speed-up therefore *widens* with problem size (from  $\sim 34\times$  at  $\sim 7\text{k}$  markers/sample to  $\sim 610\times$  at  $\sim 110\text{k}$ ) while every fit remains numerically identical (matching iteration counts, identical Viterbi paths, parameters agreeing to  $\leq 3 \times 10^{-6}$ ).